

Advanced Computing for AI

EC3 Comprehensive Exam — Study Notes

Module 4: Knowledge Representation & Logic (CS9–CS10)

Module 5: Multi-Agent Decision Making & Game Theory (CS14)

Module 6: Probabilistic Reasoning & HMM (CS11–CS12)

Module 7: Ethics & XAI (CS13)

Prof. Parthasarathy P.D. | AIMLCZG557 / AECLZG557

BITS Pilani WILP · S1-2026-27 · Open Book Exam

Exclusions: Dynamic Bayesian Networks · Prolog programs

BITS Pilani, Pilani Campus 1

Module 4

Knowledge Representation & Logic

KB Agents · Propositional Logic · CNF · Resolution · FOL · Chaining

Topics Covered:

- Knowledge-Based Agents & Wumpus World (CS9)
- Propositional Logic: Syntax, Equivalences, Entailment
- CNF Conversion Algorithm (4 steps)
- Resolution Proof by Refutation
- First-Order Logic: Quantifiers, Predicates
- Forward & Backward Chaining (CS10)

BITS Pilani, Pilani Campus 2

What is a Knowledge-Based Agent?

- Maintains a Knowledge Base (KB) — a set of sentences about the world
- TELL: add new information to the KB (STORE in KB)
- ASK: query the KB for what is true (RETRIEVE from KB)
- INFER: deduce new facts from existing KB sentences

Wumpus World — PEAS

Component	Description
Performance	+1000 gold, -1000 death, -1/step, -10/arrow
Environment	4×4 grid: hidden pits, wumpus, gold; partially observable
Actuators	Move Forward, Turn Left/Right, Grab, Shoot, Climb
Sensors	[Stench, Breeze, Glitter, Bump, Scream] — local percepts

Wumpus Inference Example

- Percept [None,None,...] at [1,1] → No pit in [1,2] or [2,1]
- Percept [Stench,...] at [1,1] → Wumpus in [1,2] or [2,1]
- Breeze at [2,1] → Pit in [2,2] or [3,1]

Propositional Syntax

- Atoms: P, Q, R (can be True or False)
- Connectives: $\neg$ (NOT) $\wedge$ (AND) $\vee$ (OR) $\rightarrow$ (IMPLIES) $\leftrightarrow$ (IFF)
- Literal: atom or its negation | Clause: disjunction of literals

Key Logical Equivalences (must memorize)

Equivalence	Form	Name
$\alpha \rightarrow \beta$	$\equiv \neg\alpha \vee \beta$	Implication Elim.
$\alpha \leftrightarrow \beta$	$\equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$	Biconditional Elim.
$\neg(\alpha \wedge \beta)$	$\equiv \neg\alpha \vee \neg\beta$	De Morgan's AND
$\neg(\alpha \vee \beta)$	$\equiv \neg\alpha \wedge \neg\beta$	De Morgan's OR
$\neg\neg\alpha$	$\equiv \alpha$	Double Negation
$\alpha \rightarrow \beta$	$\equiv \neg\beta \rightarrow \neg\alpha$	Contrapositive

Entailment

- KB $\models \alpha$ means: in every model where KB is true, α is also true
- Test: enumerate all truth assignments; if KB true $\rightarrow \alpha$ must be true

■ Trap
EXAM TIP: Entailment is "necessary truth", not just "happens to be true"

Conjunctive Normal Form (CNF)

- CNF = conjunction (AND) of clauses, each clause = disjunction (OR) of literals
- Required for Resolution algorithm — every KB sentence must be in CNF

4-Step CNF Conversion Algorithm

Step	Rule	Example
1. Elim $\leftrightarrow$	$\alpha \leftrightarrow \beta \rightarrow (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$	$A \leftrightarrow B \rightarrow (A \rightarrow B) \wedge (B \rightarrow A)$
2. Elim $\rightarrow$	$\alpha \rightarrow \beta \rightarrow \neg \alpha \vee \beta$	$(A \rightarrow B) \rightarrow (\neg A \vee B)$
3. Move $\neg$ in	De Morgan: $\neg(\alpha \wedge \beta) \rightarrow \neg \alpha \vee \neg \beta$	$\neg(A \wedge B) \rightarrow \neg A \vee \neg B$
4. Distribute $\vee$	$\alpha \vee (\beta \wedge \gamma) \rightarrow (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$	$A \vee (B \wedge C) \rightarrow (A \vee B) \wedge (A \vee C)$

Worked Example: Convert $(A \rightarrow B) \wedge \neg(B \vee C)$

- Step 1 (no $\leftrightarrow$ present, skip)
- Step 2: $(\neg A \vee B) \wedge \neg(B \vee C)$
- Step 3: $(\neg A \vee B) \wedge (\neg B \wedge \neg C)$
- Step 4: already in CNF $\rightarrow \{\neg A, B\} \wedge \{\neg B\} \wedge \{\neg C\}$

■ Trap

Step 4 (distributing $\vee$ over $\wedge$) is the most error-prone — double-check carefully

Resolution Rule & Proof by Refutation

The Resolution Rule

- If $P \vee Q$ and $\neg P \vee R$ then resolve to $Q \vee R$ (cancel complementary P , $\neg P$)
- Unit resolution: If P and $\neg P \vee Q$ then Q (Modus Ponens)

Resolution Refutation (Proof by Contradiction)

1. Negate goal: to prove α from KB, add $\neg \alpha$ to KB
2. Convert all sentences to CNF
3. Apply resolution until empty clause ■ or no new clauses
4. If ■ reached $\rightarrow$ KB ■ α ✓ (contradiction means goal is proved)

Trace Example: Prove Q from $\{P \rightarrow Q, P\}$

Step	Clause 1	Clause 2	Resolvent	Notes
1	$\{ \neg P, Q \}$	$\{ P \}$	$\{ Q \}$	Unit res. $\rightarrow Q$ proved
2	$\{ \neg Q \}$ (negated goal)	$\{ Q \}$	■ (empty!)	Contradiction $\rightarrow$ proved

★ Key Rule

Empty clause ■ = contradiction = goal proved. If no new clause can be derived $\rightarrow$ not provable

FOL Vocabulary

Element	Description	Examples
Constants	Specific named objects	John, Pilani, 3
Variables	Range over objects	x, y, z
Functions	Maps objects → object	LeftLeg(x), Father(y)
Predicates	Maps objects → T/F	King(x), Loves(x,y)
Quantifiers	Universal ∀, Existential ∃	∀x King(x), ∃x Crown(x)

Quantifier Examples

- $\forall x \text{ King}(x) \rightarrow \text{Person}(x)$ meaning: "All kings are people"
- $\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$ meaning: "Something is a crown on John"
- $\forall x \forall y \text{ Loves}(x, y)$ meaning: "Everyone loves everyone"

English → FOL Conversions

- "No dog bites John" $\rightarrow \neg \exists x \text{ Dog}(x) \wedge \text{Bites}(x, \text{John})$
- "Every student passes some exam" $\rightarrow \forall s \exists e \text{ Student}(s) \rightarrow \text{Passes}(s,e)$

Common Trap

∀ with → is standard for "all", ∃ with ∧ is standard for "there exists"
Do not mix: $\forall x P(x) \wedge Q(x) \neq \forall x P(x) \rightarrow Q(x)$ (the ∧ form is usually wrong)

Forward & Backward Chaining

Forward Chaining (Data-Driven, BFS)

- Start from known facts, fire rules whose premises are satisfied
- Repeat until goal is derived or no new fact can be inferred
- Complete for Horn clause KBs | Used in: expert systems, databases

Backward Chaining (Goal-Driven, DFS)

- Start from goal, work backward to find supporting facts
- Efficient when goal space is small | Used in: Prolog, theorem proving

Example KB: Animal Rules

Rule / Fact	Content
R1	$\text{Feathers}(x) \rightarrow \text{Bird}(x)$
R2	$\text{Fly}(x) \wedge \text{Feathers}(x) \rightarrow \text{CanFly}(x)$
Fact	$\text{Feathers}(\text{tweety}) = \text{True}, \text{Fly}(\text{tweety}) = \text{True}$
Query	$\text{CanFly}(\text{tweety})?$

- FC trace: $\text{Feathers}(\text{tweety}) \rightarrow \text{Bird}(\text{tweety})$ via R1; $\text{Fly} + \text{Feathers} \rightarrow \text{CanFly} \checkmark$
- BC trace: $\text{CanFly} \leftarrow \text{need Fly} \wedge \text{Feathers} \leftarrow \text{both in KB} \checkmark$

Note

Both give same result for Horn clauses; FC is sound&complete; BC is sound&complete too

Probabilistic Reasoning & HMM

Probability · Bayesian Networks · Markov Models · HMM · Viterbi

Topics Covered:

- Probability Fundamentals & Bayes Theorem (CS11)
- Bayesian Networks: Structure and CPTs
- Inference by Enumeration
- Markov Models & Hidden Markov Models
- Forward Algorithm (FPA / Filtering)
- Viterbi Algorithm (CS12)

Probability Fundamentals

Core Probability Rules

Rule	Formula	Meaning
Sum Rule	$P(A) = \sum P(A, B=b)$	Marginalize out B
Product Rule	$P(A, B) = P(A B) \cdot P(B)$	Joint = conditional × marginal
Bayes' Theorem	$P(H E) = P(E H) \cdot P(H) / P(E)$	Update belief given evidence
Chain Rule	$P(A, B, C) = P(A B, C) \cdot P(B C) \cdot P(C)$	Expand joint over n vars

Bayes' Theorem — Components

- **P(H|E): posterior** — belief in H given evidence E
- **P(E|H): likelihood** — probability of evidence if H is true
- **P(H): prior** — initial belief in H before evidence
- **P(E): normalisation constant** — $P(E) = \sum_h P(E|H=h) \cdot P(H=h)$

Conditional Independence

- $A \perp B \mid C$ means: given C, knowing A gives no info about B
- Key: $P(A|B, C) = P(A|C)$ when $A \perp B \mid C$

★ **Trick**
Normalise using α (alpha): $P(X|e) = \alpha \cdot P(X, e)$ where $\alpha = 1/P(e)$

What is a Bayesian Network?

- Directed Acyclic Graph (DAG): nodes = random variables, edges = direct causation
- Each node X has a CPT: $P(X \mid \text{Parents}(X))$
- Joint distribution: $P(X_1, \dots, X_n) = \prod P(X_i \mid \text{Parents}(X_i))$

Toothache–Cavity–Catch Network

Variable	Parents	CPT (sample values)
Cavity (C)	None	$P(C=T) = 0.1$
Toothache (T)	Cavity	$P(T=T C=T)=0.6, P(T=T C=F)=0.05$
Catch (X)	Cavity	$P(X=T C=T)=0.9, P(X=T C=F)=0.02$

Inference by Enumeration

- $P(\text{Cavity}|\text{Toothache}=T) = \alpha \cdot \sum_{\text{catch}} P(C, \text{Toothache}=T, \text{catch})$
- $= \alpha \cdot [P(C) \cdot P(T=T|C) \cdot \sum P(\text{catch}|C)]$
- Result: $P(C=T|T=T) \approx 0.545 \mid P(C=F|T=T) \approx 0.455$

■ **EC3 Exclusion**
Dynamic BNs are EXCLUDED from EC3 — only static BNs (fixed time-slice CPTs)

HMM Structure

- Hidden states $X_1, X_2, \dots$: not directly observable
- Observations $O_1, O_2, \dots$: observed evidence (emissions)
- Transition matrix A: $P(X_t \mid X_{t-1})$ — how states evolve
- Emission matrix B: $P(O_t \mid X_t)$ — how observations are generated
- Initial dist π : $P(X_1)$ — starting state probabilities

Weather–Pressure Example

	Sunny (next)	Rainy (next)
Sunny (now)	0.8	0.2
Rainy (now)	0.4	0.6

State	P(High pressure)	P(Low pressure)
Sunny	0.9	0.1
Rainy	0.2	0.8

HMM Tasks

- Filtering (FPA): $P(X_t \mid O_{1:t})$ — most likely current state given obs so far
- Smoothing: $P(X_k \mid O_{1:T}) \ k < T$ — refine past state estimate with future obs
- Decoding (Viterbi): most likely STATE SEQUENCE given observations

Viterbi: Most Likely State Sequence

- **Goal:** find $\text{argmax } P(X_1, X_2, \dots, X_n \mid O_{1:n})$
- Initialise: $\delta_1(j) = \pi(j) \cdot b_j(O_1)$ — prob of best path ending in state j at $t=1$
- Recurse: $\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(O_t)$
- Backtrack: $\psi_t(j) = \text{argmax}_i [\delta_{t-1}(i) \cdot a_{ij}]$ — which previous state was best

Viterbi Trace: Observations = [High, Low] (Weather HMM)

t	State j	$\delta_t(j)$	$\psi_t(j)$	Computation
1	Sunny	0.360	—	$\pi(S) \cdot P(\text{High} S) = 0.4 \cdot 0.9$
1	Rainy	0.040	—	$\pi(R) \cdot P(\text{High} R) = 0.2 \cdot 0.2$
2	Sunny	0.026	Sunny	$\max(0.36 \cdot 0.8, 0.04 \cdot 0.4) \cdot P(\text{Low} S) = 0.288 \cdot 0.1$
2	Rainy	0.189	Sunny	$\max(0.36 \cdot 0.2, 0.04 \cdot 0.6) \cdot P(\text{Low} R) = 0.072 \cdot 0.8$ — BEST

- Best path: backtrack from $\delta_2 \text{ max (Rainy)} \rightarrow \psi_2(\text{Rainy}) = \text{Sunny} \rightarrow [\text{Sunny, Rainy}]$

★ Key Difference

Viterbi uses MAX not SUM; FPA (filtering) uses SUM. Viterbi = MAX·PRODUCT path

Forward Algorithm (Filtering / FPA)

Forward Probability Algorithm

- Computes $\alpha_t(j) = P(O_{1:t}, X_t=j)$ — joint prob of obs AND being in state j
- Initialise: $\alpha_1(j) = \pi(j) \cdot b_j(O_1)$
- Recurse: $\alpha_t(j) = b_j(O_t) \cdot \sum_i [\alpha_{t-1}(i) \cdot a_{ij}]$
- Output: $P(X_t=j \mid O_{1:t}) = \alpha_t(j) / \sum_k \alpha_t(k)$ (normalise)

FPA Trace: Observations = [High, Low]

t	State j	$\alpha_t(j)$	Computation
1	Sunny	0.360	$\pi(S) \cdot P(\text{High} S) = 0.4 \times 0.9$
1	Rainy	0.040	$\pi(R) \cdot P(\text{High} R) = 0.2 \times 0.2$
2	Sunny	0.026	$P(\text{Low} S) \cdot [\alpha_1(S) \cdot A_{SS} + \alpha_1(R) \cdot A_{RS}] = 0.1 \cdot (0.36 \cdot 0.8 + 0.04 \cdot 0.4)$
2	Rainy	0.189	$P(\text{Low} R) \cdot [\alpha_1(S) \cdot A_{SR} + \alpha_1(R) \cdot A_{RR}] = 0.8 \cdot (0.36 \cdot 0.2 + 0.04 \cdot 0.6)$

- Normalise: $P(\text{Rainy}|\text{obs}) = 0.189 / (0.026 + 0.189) = 0.879 \rightarrow$ most likely Rainy

■ Compare

FPA sums over all paths (total probability); Viterbi maximises over the single best path

Module 5

Multi-Agent Decision Making

Game Theory · Nash Equilibrium · Cooperative Games · Shapley Value

Topics Covered:

- Multi-Agent Environments & Normal-Form Games (CS14)
- Payoff Matrices, Dominant Strategies
- Prisoner's Dilemma & Nash Equilibrium
- Battle of the Sexes: Multiple Nash Equilibria
- IEDS: Iterated Elimination of Dominated Strategies
- Cooperative Games & Shapley Value
- Collective Decision Making: Borda, Condorcet, Veto

Multi-Agent Environments & Game Theory

Module 5

Multi-Agent System Taxonomy

Type	Description	Example
Fully cooperative	All agents share same goal	Robot soccer team
Fully competitive	Zero-sum: one wins, other loses	Chess, Poker
Mixed	Self-interested; may cooperate	Market, negotiation

Key Concepts

- **Strategy:** a complete plan specifying action for every possible situation
- Pure strategy: deterministic choice | Mixed strategy: randomised choice
- Payoff: utility an agent receives for a combination of strategies

Normal Form Game

- Represent by payoff matrix: rows = Player 1 strategies, cols = Player 2
- Entry (a, b): Player 1 gets a, Player 2 gets b

EC3 Focus
For EC3: focus on pure strategies Nash Equilibrium, dominant strategies, Shapley

Prisoner's Dilemma — Classic Payoff Matrix

	P2: Cooperate (C)	P2: Defect (D)
P1: Cooperate (C)	(-1, -1)	(-3, 0)
P1: Defect (D)	(0, -3)	(-2, -2) ← NE

- Dominant strategy for each player: Defect — regardless of other player
- Nash Equilibrium at (D, D): neither player benefits by unilaterally changing

Finding Nash Equilibrium (Best-Response Method)

1. For each column (P2 strategy), mark P1's best response with ★
2. For each row (P1 strategy), mark P2's best response with ★
3. Nash Equilibrium: cell where BOTH payoffs are starred (★★)

Battle of the Sexes — Multiple Nash Equilibria

	P2: Opera (O)	P2: Football (F)
P1: Opera (O)	(2, 1) ← NE1	(0, 0)
P1: Football (F)	(0, 0)	(1, 2) ← NE2

★ Key Fact

A game can have 0, 1, or multiple pure Nash Equilibria (BoS has 2 pure NE)

IEDS & Shapley Value

IEDS: Iterated Elimination of Dominated Strategies

- Strictly dominated: strategy s is dominated if $\exists s'$ with higher payoff in ALL cases
- Remove dominated strategies iteratively; surviving strategies = rationalizable
- If one strategy survives per player → unique Nash Equilibrium

Shapley Value — Fair Payoff in Cooperative Games

- Unique fair division satisfying: efficiency, symmetry, null player, linearity
- $\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{[S]!(|N|-|S|-1)!}{|N|!} \cdot [v(S \cup \{i\}) - v(S)]$
- Interpretation: i's Shapley value = average marginal contribution over all orderings

Shapley Example: 3 Programmers A,B,C

Ordering	MC(A)	MC(B)	MC(C)
A,B,C	40	20	40
A,C,B	40	20	40
B,A,C	60	0	40
B,C,A	60	0	40
C,A,B	60	0	40
C,B,A	60	0	40
Average ϕ	56.7	6.7	36.7

★ Check

Efficiency: $\phi A + \phi B + \phi C = v(N)$ — total value is fully distributed

Social Choice: Aggregating Preferences

Given n voters each with preferences over m alternatives — how to decide?

Voting Methods Comparison

Method	Rule	Pros / Cons
Plurality	Most 1st-place votes wins	Simple; ignores lower prefs
Borda Count	Points: m-1 for 1st, m-2 for 2nd...	Considers full ranking
Condorcet	Wins pairwise vs all others	Fair; may not exist
Veto	Eliminate least-liked alternatives	Prevents worst outcomes

Borda Count Worked Example

Alt	V1 (2,1,0)	V2 (2,1,0)	V3 (2,1,0)	Total
A	2 (1st)	0 (3rd)	2 (1st)	4
B	1 (2nd)	1 (2nd)	1 (2nd)	3
C	0 (3rd)	2 (1st)	0 (3rd)	2
Winner	A wins with 4 pts			

Theorem
Arrow's Impossibility: no voting system satisfies ALL fairness axioms simultaneously

EC3 Key Formulas Reference

Logic & KB

- Resolution: $\{P \vee Q\} \wedge \{\neg P \vee R\} \rightarrow \{Q \vee R\}$ | Empty clause $\blacksquare$ = proved
- CNF: $\text{Elim} \leftrightarrow \rightarrow \text{Elim} \rightarrow \rightarrow \text{Move } \neg \text{ inward} \rightarrow \text{Distribute } \vee \text{ over } \wedge$

Bayesian Networks

- Joint: $P(X_1 \dots X_n) = \prod P(X_i | \text{Parents}(X_i))$
- Inference: $P(X|e) = \alpha \cdot \sum_{\text{hidden}} P(X, e, \text{hidden})$

HMM Algorithms

Algorithm	Formula	Use
FPA (Filter)	$\alpha_t(j) = b_j(O_t) \cdot \sum_i [\alpha_{t-1}(i) \cdot a_{ij}]$	$P(\text{current state} \mid \text{obs})$
Viterbi	$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(O_t)$	Best state sequence

Game Theory

- Nash Eq: (s^*, t^*) where $u_1(s^*, t^*) \geq u_1(s, t^*) \forall s$ AND $u_2(s^*, t^*) \geq u_2(s^*, t) \forall t$
- Shapley: $\phi_i = \sum_S [|S|!(n-|S|-1)!/n!] \cdot [v(S \cup i) - v(S)]$

Borda
Borda score for m alternatives: 1st place = m-1 pts, 2nd = m-2, ..., last = 0

EC3 Exam: Open Book — 13 Sep 2026, 1:00 PM

- Covers Modules 1–7 (Modules 1–3 from main ACI portal)
- Exclusions: Dynamic Bayesian Networks, Prolog programs

Priority Topics by Module

Module	High-Value Topics	Likely Q type
M4: Logic	CNF conversion, Resolution proof, FOL quantifiers	Trace / Prove
M5: Games	Nash Eq (find + verify), Shapley worked calc, Borda	Calculate
M6: Prob	Viterbi step-by-step, BN CPT inference, FPA	Trace / Compute
M7: Ethics	XAI types: LIME, SHAP, counterfactuals (post-Aug30)	Explain

Exam Traps to Avoid

- Forgetting to distribute $\vee$ over $\wedge$ in CNF step 4
- Confusing FPA (sum) with Viterbi (max) in HMM
- Using $\forall x P(x) \wedge Q(x)$ instead of $\forall x P(x) \rightarrow Q(x)$ in FOL
- Missing the normalisation step in BN inference
- Forgetting to check ALL orderings in Shapley ($n!$ orderings)

■ Open Book Tip

Open book: prepare a 1-page formula sheet with CNF steps, Viterbi/FPA formulas, Shapley formula